

Techastra '26 — Code Rescue

OFFICIAL DEBUGGING CHAMPIONSHIP QUESTION BANK (CALIBRATED & STREAMLINED)

Authoritative Competition Blueprint: Liberal Difficulty | Exactly 2 Bugs (R1), 5 Bugs (R2), 10 Bugs (R3) | Streamlined & Concise Code

Round	Stage Title	Questions	Bugs / Q	Marks / Q	Total Marks	Duration	Cutoff
Round 1	Bug Hunt (Easy / Fast)	10 Questions	Exactly 2	1 Mark	10 Marks	15 Mins	≥ 5 Marks
Round 2	Logic Breaker (Liberal)	5 Questions	Exactly 5	4 Marks	20 Marks	20 Mins	≥ 10 Marks
Round 3	Code Rescue Finale	1 Grand System	Exactly 10	5 Marks	5 Marks	25 Mins	Leaderboard
TOTAL	All 3 Rounds	16 Challenges	55 Total Bugs	-	35 Marks	60 Mins	Top Rankers

ROUND 1 — BUG HUNT

Format: 10 Questions | **Target:** Exactly 2 Bugs per Question | **Difficulty:** Liberal / Accessible (Beginner Friendly) | **Score:** 1 Mark each (Total: 10 Marks) | **Duration:** 15 Minutes

Q1. Student Marks Average (List Iteration & Arithmetic Division)

Work Order: Calculate the arithmetic average of marks in a student's mark list, returned as a float rounded to 2 decimal places.

Function Signature: def calculate_average(marks: list[int]) -> float

Verification Sample: Input: [78, 85, 92, 88] → Expected Output: **85.75**

Buggy Starter Code (2 Errors Injected):

```
1 def calculate_average(marks):
2     total = 0
3     for i in range(len(marks) - 1):
4         total += marks[i]
5     return total // len(marks)
```

Identified Errors & Explanation:

- Bug 1 (Line 3):** Loop range uses range(len(marks) - 1), which excludes the last element in the list.
- Bug 2 (Line 5):** Uses integer floor division // instead of floating-point division /, discarding decimal accuracy.

Verified Correct Solution:

```
1 def calculate_average(marks):
2     if not marks:
3         return 0.0
4     total = 0
5     for i in range(len(marks)):
6         total += marks[i]
7     return round(total / len(marks), 2)
```

Test Baseline: Visible: [78, 85, 92, 88] -> 85.75 | [100, 90] -> 95.0 | Hidden: [45, 55, 65, 75] -> 60.0 | [10] -> 10.0

Q2. Palindrome String Checker (String Slicing & Case Sensitivity)

Work Order: Determine whether an input string reads identically forwards and backwards, ignoring character case.

Function Signature: def is_palindrome(text: str) -> bool

Verification Sample: Input: 'Racecar' → Expected Output: **True**

Buggy Starter Code (2 Errors Injected):

```
1 def is_palindrome(text):
2     reversed_text = text[::-1]
3     if text == reversed_text:
4         return False
5     return True
```

Identified Errors & Explanation:

- **Bug 1 (Line 2):** Does not call `.lower()` on text, causing uppercase letters to fail palindrome symmetry (e.g. 'Racecar').
- **Bug 2 (Lines 3-5):** Returns False when `text == reversed_text` and True otherwise (inverted boolean logic).

Verified Correct Solution:

```
1 def is_palindrome(text):
2     cleaned = text.lower()
3     return cleaned == cleaned[::-1]
```

Test Baseline: Visible: 'Racecar' -> True | 'Python' -> False | Hidden: 'Madam' -> True | '12321' -> True | 'techastra' -> False

Q3. Discount & Final Price Calculator (Percentage Arithmetic & Price Deduction)

Work Order: Calculate the final payable price after applying a percentage discount to an item's original price.

Function Signature: `def calculate_final_price(price: float, discount_percent: float) -> float`

Verification Sample: Input: price = 1200.0, discount_percent = 15.0 → Expected Output: **1020.0**

Buggy Starter Code (2 Errors Injected):

```
1 def calculate_final_price(price, discount_percent):
2     discount = price * (discount_percent / 10)
3     final_price = price + discount
4     return round(final_price, 2)
```

Identified Errors & Explanation:

- **Bug 1 (Line 2):** Divides `discount_percent` by 10 instead of 100, inflating discount by 10x.
- **Bug 2 (Line 3):** Adds discount to price instead of subtracting it (price surcharge instead of discount).

Verified Correct Solution:

```
1 def calculate_final_price(price, discount_percent):
2     discount = price * (discount_percent / 100.0)
3     final_price = price - discount
4     return round(final_price, 2)
```

Test Baseline: Visible: (1200.0, 15.0) -> 1020.0 | (500.0, 10.0) -> 450.0 | Hidden: (250.0, 0.0) -> 250.0 | (199.99, 50.0) -> 100.0

Q4. Find Maximum in Integer List (Initialization Boundary & Comparison Operator)

Work Order: Find and return the maximum integer value in a non-empty list of integers, including lists of negative numbers.

Function Signature: `def find_maximum(numbers: list[int]) -> int`

Verification Sample: Input: [-12, -5, -20, -3] → Expected Output: **-3**

Buggy Starter Code (2 Errors Injected):

```
1 def find_maximum(numbers):
2     maximum = 0
3     for num in numbers:
4         if num < maximum:
5             maximum = num
6     return maximum
```

Identified Errors & Explanation:

- **Bug 1 (Line 2):** Initializes `maximum = 0`, which incorrectly yields 0 when all numbers in the list are negative.
- **Bug 2 (Line 4):** Uses `if num < maximum` (minimum search) instead of `num > maximum`.

Verified Correct Solution:

```

1 def find_maximum(numbers):
2     maximum = numbers[0]
3     for num in numbers:
4         if num > maximum:
5             maximum = num
6     return maximum

```

Test Baseline: Visible: [-12, -5, -20, -3] -> -3 | [10, 45, 23] -> 45 | Hidden: [-100] -> -100 | [5, 5, 5] -> 5 | [-1, 0, 1] -> 1

Q5. Count Even Numbers (Loop Variable Scope & Modulo Condition)

Work Order: Count how many even integers are present in a given list of integers.

Function Signature: def count_evens(numbers: list[int]) -> int

Verification Sample: Input: [1, 2, 3, 4, 6, 7] → Expected Output: 3

Buggy Starter Code (2 Errors Injected):

```

1 def count_evens(numbers):
2     for n in numbers:
3         count = 0
4         if n % 2 == 1:
5             count += 1
6     return count

```

Identified Errors & Explanation:

- **Bug 1 (Line 3):** Counter count = 0 is reinitialized inside the loop, resetting the tally on every iteration.
- **Bug 2 (Line 4):** Checks n % 2 == 1, which counts odd numbers instead of even numbers.

Verified Correct Solution:

```

1 def count_evens(numbers):
2     count = 0
3     for n in numbers:
4         if n % 2 == 0:
5             count += 1
6     return count

```

Test Baseline: Visible: [1, 2, 3, 4, 6, 7] -> 3 | [2, 4, 6] -> 3 | Hidden: [1, 3, 5] -> 0 | [] -> 0 | [0, 8, -2] -> 3

Q6. Celsius to Fahrenheit Converter (Arithmetic Order of Operations & Formula Constants)

Work Order: Convert a given temperature in Celsius to Fahrenheit using the formula $(C * 9/5) + 32$, rounded to 2 decimal places.

Function Signature: def celsius_to_fahrenheit(celsius: float) -> float

Verification Sample: Input: 25.0 → Expected Output: 77.0

Buggy Starter Code (2 Errors Injected):

```

1 def celsius_to_fahrenheit(celsius):
2     fahrenheit = (celsius * (5 / 9)) - 32
3     return round(fahrenheit, 2)

```

Identified Errors & Explanation:

- **Bug 1 (Line 2):** Inverts temperature ratio to 5 / 9 instead of 9 / 5.
- **Bug 2 (Line 2):** Subtracts 32 instead of adding 32 in the conversion equation.

Verified Correct Solution:

```

1 def celsius_to_fahrenheit(celsius):
2     fahrenheit = (celsius * (9.0 / 5.0)) + 32
3     return round(fahrenheit, 2)

```

Test Baseline: Visible: 25.0 -> 77.0 | 0.0 -> 32.0 | Hidden: 100.0 -> 212.0 | -40.0 -> -40.0 | 37.0 -> 98.6

Q7. Word Frequency Counter (Dictionary Mutation & Case Normalization)

Work Order: Count the frequency of each unique word in a list of words, normalized to lowercase.

Function Signature: `def count_words(words: list[str]) -> dict[str, int]`

Verification Sample: Input: ['apple', 'Banana', 'APPLE', 'banana', 'apple'] → Expected Output: {'apple': 3, 'banana': 2}

Buggy Starter Code (2 Errors Injected):

```
1 def count_words(words):
2     freq = {}
3     for word in words:
4         if word in freq:
5             freq[word] = 1
6         else:
7             freq[word] = 1
8     return freq
```

Identified Errors & Explanation:

- **Bug 1 (Line 3):** Does not convert words with `.lower()`, causing uppercase variants to be treated as distinct keys.
- **Bug 2 (Line 5):** Sets `freq[word] = 1` when the key is already found instead of incrementing `freq[word] += 1`.

Verified Correct Solution:

```
1 def count_words(words):
2     freq = {}
3     for word in words:
4         w = word.lower()
5         freq[w] = freq.get(w, 0) + 1
6     return freq
```

Test Baseline: Visible: ['apple', 'Banana', 'APPLE', 'banana', 'apple'] -> {'apple': 3, 'banana': 2} | Hidden: ['a', 'b', 'A'] -> {'a': 2, 'b': 1} | [] -> {}

Q8. Factorial Calculator (Base Case Initialization & Range Boundary)

Work Order: Calculate the factorial of a non-negative integer n ($0! = 1$, $5! = 120$).

Function Signature: `def factorial(n: int) -> int`

Verification Sample: Input: 5 → Expected Output: 120

Buggy Starter Code (2 Errors Injected):

```
1 def factorial(n):
2     if n == 0:
3         return 0
4     fact = 1
5     for i in range(1, n):
6         fact *= i
7     return fact
```

Identified Errors & Explanation:

- **Bug 1 (Line 3):** Base case `n == 0` returns 0 instead of 1 ($0!$ is mathematically 1).
- **Bug 2 (Line 5):** Range loop `range(1, n)` stops at `n - 1`, excluding the multiplicand `n` itself.

Verified Correct Solution:

```
1 def factorial(n):
2     if n == 0 or n == 1:
3         return 1
4     fact = 1
5     for i in range(1, n + 1):
6         fact *= i
7     return fact
```

Test Baseline: Visible: 5 -> 120 | 0 -> 1 | Hidden: 1 -> 1 | 4 -> 24 | 6 -> 720

Q9. Reverse Sublist by Index Range (Slicing Index Inclusivity & Direction Step)

Work Order: Reverse a portion of a list between indices start and end (inclusive) and return the resulting list.

Function Signature: `def reverse_sublist(lst: list, start: int, end: int) -> list`

Verification Sample: Input: `lst = [1, 2, 3, 4, 5, 6]`, `start = 1`, `end = 4` → Expected Output: `[1, 5, 4, 3, 2, 6]`

Buggy Starter Code (2 Errors Injected):

```
1 def reverse_sublist(lst, start, end):
2     sub = lst[start:end]
3     sub_rev = sub[::-1]
4     return lst[:start] + sub_rev + lst[end + 1:]
```

Identified Errors & Explanation:

- **Bug 1 (Line 2):** Slice `lst[start:end]` excludes index end; must be `lst[start:end + 1]` for inclusivity.
- **Bug 2 (Line 3):** Slicing step `::1` preserves forward direction; must be `::-1` to reverse.

Verified Correct Solution:

```
1 def reverse_sublist(lst, start, end):
2     sub = lst[start:end + 1]
3     sub_rev = sub[::-1]
4     return lst[:start] + sub_rev + lst[end + 1:]
```

Test Baseline: Visible: `([1, 2, 3, 4, 5, 6], 1, 4) -> [1, 5, 4, 3, 2, 6]` | Hidden: `([10, 20, 30], 0, 2) -> [30, 20, 10]` | `([1, 2, 3], 1, 1) -> [1, 2, 3]`

Q10. Vowel Counter (Character Membership & Accumulator Scope)

Work Order: Count the total number of vowels (A, E, I, O, U, case-insensitive) in a given string.

Function Signature: `def count_vowels(text: str) -> int`

Verification Sample: Input: `'Education'` → Expected Output: `5`

Buggy Starter Code (2 Errors Injected):

```
1 def count_vowels(text):
2     vowels = "aeiou"
3     for ch in text:
4         count = 0
5         if ch in vowels:
6             count += 1
7     return count
```

Identified Errors & Explanation:

- **Bug 1 (Line 4):** Counter `count = 0` is reset inside the loop body on every character inspection.
- **Bug 2 (Line 2):** Vowel target string contains only lowercase 'aeiou', missing uppercase vowels like 'E' in 'Education'.

Verified Correct Solution:

```
1 def count_vowels(text):
2     count = 0
3     vowels = "aeiouAEIOU"
4     for ch in text:
5         if ch in vowels:
6             count += 1
7     return count
```

Test Baseline: Visible: `'Education' -> 5` | `'rhythm' -> 0` | Hidden: `'AEIOU' -> 5` | `'Python Programming' -> 4` | `" -> 0`

ROUND 2 — LOGIC BREAKER (STREAMLINED)

Format: 5 Questions | **Target:** Exactly 5 Bugs per Question | **Difficulty:** Liberal / Moderate (Calibrated for 4 Mins/Q) | **Score:** 4 Marks each (Total: 20 Marks) | **Duration:** 20 Minutes

Q1. Electricity Tariff Slabs & Surcharge Calculator (Utility Billing & Multi-Tier Slabs)

Calculate electricity bill: first 100 units @ ₹2.50/unit; units above 100 @ ₹4.50/unit. Fixed meter charge: ₹50 for 'domestic', ₹150 for 'commercial'. If energy bill exceeds ₹1000, a 5% surcharge is added to the energy bill before adding fixed charge.

Work Order:

Function Signature: `def calculate_electricity_bill(units: int, meter_type: str) -> float`

Input: units = 250, meter_type = 'domestic' → Expected Output: **975.0** (Energy: $100 \times 2.5 + 150 \times 4.5 = 925.0$,

Verification Sample: Fixed: 50.0)

Buggy Starter Code (5 Errors Injected):

```
1 def calculate_electricity_bill(units, meter_type):
2     # BUG 1: strict < 100 ignores 100 unit boundary
3     # BUG 2: multiplies entire units by 4.5 instead of marginal units
4     energy = units * 2.5 if units < 100 else 100 * 2.5 + units * 4.5
5     # BUG 3: surcharge calculated on unit count instead of energy bill
6     surcharge = units * 0.05 if energy > 1000 else 0.0
7     # BUG 4: checks 'comm' instead of 'commercial'
8     fixed = 150.0 if meter_type.lower() == 'comm' else 50.0
9     # BUG 5: subtracts fixed charge instead of adding it
10    total = energy + surcharge - fixed
11    return round(total, 2)
```

Identified Errors & Explanation:

- **Bug 1 (Line 4):** Slab boundary uses `units < 100` instead of `units <= 100`.
- **Bug 2 (Line 4):** Computes `units * 4.5` on full consumption instead of marginal units `(units - 100) * 4.5`.
- **Bug 3 (Line 6):** Computes 5% surcharge on consumption count `units * 0.05` instead of monetary energy `energy * 0.05`.
- **Bug 4 (Line 8):** String lookup checks token `'comm'` instead of expected parameter `'commercial'`.
- **Bug 5 (Line 10):** Fixed meter charge is deducted `- fixed` instead of added `+ fixed`.

Verified Correct Solution:

```
1 def calculate_electricity_bill(units, meter_type):
2     energy = units * 2.5 if units <= 100 else 100 * 2.5 + (units - 100) * 4.5
3     surcharge = energy * 0.05 if energy > 1000 else 0.0
4     fixed = 150.0 if meter_type.lower() == 'commercial' else 50.0
5     total = energy + surcharge + fixed
6     return round(total, 2)
```

Test Baseline: Visible: (250, 'domestic') -> 975.0 | (50, 'domestic') -> 175.0 | Hidden: (300, 'commercial') -> 1357.5 | (100, 'domestic') -> 300.0

Q2. Student Academic Evaluation & Ranking Engine (Academic Analytics & Categorization)

Calculate total marks, average, grade ('A' if average ≥ 75 else 'B'), and count of subjects with distinction (≥ 75). If any subject has marks < 40 , grade is immediately 'F'.

Work Order:

Function Signature: `def evaluate_student(marks: list[int]) -> dict`

Verification Sample: Input: [85, 90, 78, 65, 82] → Expected Output: **{'total': 400, 'average': 80.0, 'grade': 'A', 'distinction_count': 4}**

Buggy Starter Code (5 Errors Injected):

```

1 def evaluate_student(marks):
2     # BUG 1: omits last mark from sum
3     total = sum(marks[:-1])
4     average = round(total / len(marks), 2)
5     # BUG 2: inverted fail rule: flags fail if mark > 40
6     failed = any(m > 40 for m in marks)
7     # BUG 3: strict distinction check > 75 instead of >= 75
8     distinction_count = sum(1 for m in marks if m > 75)
9     # BUG 4: inverted grade cutoff: assigns 'C' for high average and 'A' for low
10    grade = "F" if failed else ("A" if average < 75 else "B")
11    # BUG 5: typo in key 'avg' instead of 'average'
12    return {
13        "total": total,
14        "avg": average,
15        "grade": grade,
16        "distinction_count": distinction_count
17    }

```

Identified Errors & Explanation:

- **Bug 1 (Line 3):** Slice marks[:-1] excludes the last mark from the total.
- **Bug 2 (Line 6):** Inverted passing condition: m > 40 flags passing students as failed.
- **Bug 3 (Line 8):** Distinction check uses strict inequality m > 75 instead of m >= 75.
- **Bug 4 (Line 10):** Ternary condition inverted: assigns 'A' when average < 75.
- **Bug 5 (Line 14):** Returned dictionary uses truncated key 'avg' instead of 'average'.

Verified Correct Solution:

```

1 def evaluate_student(marks):
2     total = sum(marks)
3     average = round(total / len(marks), 2)
4     failed = any(m < 40 for m in marks)
5     distinction_count = sum(1 for m in marks if m >= 75)
6     grade = "F" if failed else ("A" if average >= 75 else "B")
7     return {
8         "total": total,
9         "average": average,
10        "grade": grade,
11        "distinction_count": distinction_count
12    }

```

Test Baseline: Visible: [85, 90, 78, 65, 82] -> {'total': 400, 'average': 80.0, 'grade': 'A', 'distinction_count': 4} | Hidden: [95, 30, 80] -> grade: 'F'

Q3. E-Commerce Shopping Cart & Checkout Engine (Retail Cart Processing & Tiered Promotions)

Process a cart of items (each item has 'name', 'price', 'qty'). Ignore items with qty <= 0. Subtotal = sum(price * qty). Coupon 'SAVE10' gives 10% off. Members get flat ₹100 discount. Apply 5% GST on net amount. Shipping is ₹50 if net < 500, else FREE (₹0).

Work Order:**Function Signature:**

def calculate_cart_checkout(items: list[dict], coupon: str, is_member: bool) -> dict

Verification Sample:

Input: items = [{'price': 800, 'qty': 2}, {'price': 300, 'qty': 1}], coupon = 'SAVE10', is_member = True → Expected Output: {'subtotal': 1900.0, 'discount': 290.0, 'final_total': 1690.5}

Buggy Starter Code (5 Errors Injected):

```

1 def calculate_cart_checkout(items, coupon, is_member):
2     # BUG 1: does not skip items with qty <= 0
3     # BUG 2: adds price + qty instead of multiplying
4     subtotal = sum(item["price"] + item["qty"] for item in items)
5     discount = subtotal * 0.10 if coupon == "SAVE10" else 0.0
6     # BUG 3: member discount deducted from subtotal instead of added to discount
7     if is_member:
8         subtotal -= 100.0
9     net = max(0.0, subtotal - discount)
10    # BUG 4: tax computed on subtotal instead of net amount
11    tax = round(subtotal * 0.05, 2)
12    # BUG 5: inverted shipping condition: charges 50 when net >= 500
13    shipping = 50.0 if net >= 500.0 else 0.0
14    return {
15        "subtotal": round(subtotal, 2),
16        "discount": round(discount, 2),
17        "final_total": round(net + tax + shipping, 2)
18    }

```

Identified Errors & Explanation:

- **Bug 1 (Line 4):** Does not filter invalid lines with qty <= 0.
- **Bug 2 (Line 4):** Computes line total as addition price + qty instead of multiplication price * qty.
- **Bug 3 (Line 8):** Mutates base subtotal directly instead of accumulating into discount.
- **Bug 4 (Line 11):** Computes GST on original subtotal instead of net discounted amount.
- **Bug 5 (Line 13):** Free shipping conditional is inverted: charges ₹50 shipping on high orders (>= 500).

Verified Correct Solution:

```

1 def calculate_cart_checkout(items, coupon, is_member):
2     subtotal = sum(item["price"] * item["qty"] for item in items if item.get("qty", 0) > 0)
3     discount = subtotal * 0.10 if coupon == "SAVE10" else 0.0
4     if is_member and subtotal > 0:
5         discount += 100.0
6     discount = min(discount, subtotal)
7     net = subtotal - discount
8     tax = round(net * 0.05, 2)
9     shipping = 0.0 if (net >= 500.0 or net == 0.0) else 50.0
10    return {
11        "subtotal": round(subtotal, 2),
12        "discount": round(discount, 2),
13        "final_total": round(net + tax + shipping, 2)
14    }

```

Test Baseline: Visible: 2 items @ 800x2 + 300x1, 'SAVE10', True -> final_total: 1690.5 | Hidden: 1 item @ 200x1, 'NONE', False -> final_total: 260.0

Q4. Matrix Diagonals & Boundary Cell Aggregator (2D Matrix Processing & Geometric Traversal)

Given an n x n square matrix of integers, calculate: 1. Main diagonal sum (row == col); 2. Secondary diagonal sum (row + col == n - 1); 3. Boundary sum (all outer border cells, counted once). Return dictionary with 'main_diag', 'sec_diag', and 'boundary'.

Work Order:

Function Signature: def matrix_diagonals_and_boundary(matrix: list[list[int]]) -> dict

Verification Sample: Input: [[1, 2, 3], [4, 5, 6], [7, 8, 9]] → Expected Output: {'main_diag': 15, 'sec_diag': 15, 'boundary': 40}

Buggy Starter Code (5 Errors Injected):

```

1 def matrix_diagonals_and_boundary(matrix):
2     n = len(matrix)
3     main_diag, sec_diag = 0, 0
4     # BUG 1: loop range(n - 1) ignores last row
5     for i in range(n - 1):
6         # BUG 2: main_diag reset inside loop
7         main_diag = 0
8         main_diag += matrix[i][i]
9         # BUG 3: IndexError: matrix[i][n - i] instead of n - i - 1
10        sec_diag += matrix[i][n - i]
11    # BUG 4: boundary double-counts corners
12    boundary = sum(matrix[0]) + sum(matrix[n - 1]) + sum(matrix[r][0] + matrix[r][n - 1] for r in range(n))
13    # BUG 5: swapped keys for main and secondary diagonals
14    return {"main_diag": sec_diag, "sec_diag": main_diag, "boundary": boundary}

```

Identified Errors & Explanation:

- **Bug 1 (Line 5):** Loop runs range(n - 1), omitting the final matrix row.
- **Bug 2 (Line 7):** Reinitializes accumulator main_diag = 0 inside loop body.
- **Bug 3 (Line 9):** Computes index matrix[i][n - i], throwing fatal IndexError.
- **Bug 4 (Line 11):** Boundary sum naively adds border rows and border columns, double-counting all four corners.
- **Bug 5 (Line 13):** Output dictionary swaps assignment of main_diag and sec_diag.

Verified Correct Solution:

```

1 def matrix_diagonals_and_boundary(matrix):
2     n = len(matrix)
3     main_diag = sum(matrix[i][i] for i in range(n))
4     sec_diag = sum(matrix[i][n - i - 1] for i in range(n))
5     boundary = sum(matrix[r][c] for r in range(n) for c in range(n) if r == 0 or r == n - 1 or c == 0 or c == n - 1)
6     return {"main_diag": main_diag, "sec_diag": sec_diag, "boundary": boundary}

```

Test Baseline: Visible: 3x3 matrix [[1,2,3],[4,5,6],[7,8,9]] -> {'main_diag': 15, 'sec_diag': 15, 'boundary': 40} | Hidden: 1x1 matrix [[5]] -> boundary: 5

Q5. Password & Security Token Validator (Authentication & Input Sanitization)

Validate user security credentials. Password must be length 8-20, contain uppercase, lowercase, digit, special character from '!@#%\$^&*', and no whitespace. Token must start with 'TK-', length 8, with hex characters after prefix. Both must be valid for is_authorized.

Work Order:

Function Signature: def validate_credentials(password: str, token: str) -> dict

Input: password = 'Secure@Pass123', token = 'TK-4F2A1' → Expected Output: {'password_valid': True,

Verification Sample: 'token_valid': True, 'is_authorized': True}

Buggy Starter Code (5 Errors Injected):

```

1 def validate_credentials(password, token):
2     # BUG 1: length condition checks <= 8 instead of >= 8
3     len_ok = len(password) <= 8 and len(password) <= 20
4     has_char = (any(c.isupper() for c in password) and any(c.islower() for c in password)
5                 and any(c.isdigit() for c in password))
6     # BUG 2: specials string missing @$
7     has_spec = any(c in "!%&*" for c in password)
8     # BUG 3: inverted space check: requires space instead of forbidding
9     has_no_space = ' ' in password
10    pw_valid = len_ok and has_char and has_spec and has_no_space
11    # BUG 4: token prefix checks 'TK' (len 2) instead of 'TK-' (len 3)
12    tk_valid = token.startswith("TK") and len(token) == 8 and all(c in "0123456789abcdefABCDEF" for c in token[2:])
13    # BUG 5: authorization uses OR instead of AND
14    return {"password_valid": pw_valid, "token_valid": tk_valid, "is_authorized": pw_valid or tk_valid}

```

Identified Errors & Explanation:

- **Bug 1 (Line 3):** Password length condition checks len(password) <= 8 instead of len(password) >= 8.
- **Bug 2 (Line 6):** Special character string definition omits @, #, and \$.
- **Bug 3 (Line 8):** Whitespace check inverted: ' ' in password requires a space instead of forbidding.

- **Bug 4 (Line 11):** Prefix check verifies 'TK' without the hyphen 'TK-'.
- **Bug 5 (Line 13):** Master authorization flag combines credentials using pw_valid or tk_valid instead of strict and.

Verified Correct Solution:

```
1 def validate_credentials(password, token):
2     len_ok = 8 <= len(password) <= 20
3     has_char = (any(c.isupper() for c in password) and any(c.islower() for c in password)
4                 and any(c.isdigit() for c in password))
5     has_spec = any(c in "!@#$$%^&*" for c in password)
6     has_no_space = ' ' not in password
7     pw_valid = bool(len_ok and has_char and has_spec and has_no_space)
8     tk_valid = (token.startswith("TK-") and len(token) == 8 and
9                all(c in "0123456789abcdefABCDEF" for c in token[3:]))
10    return {"password_valid": pw_valid, "token_valid": tk_valid, "is_authorized": pw_valid and tk_valid}
```

Test Baseline: Visible: ('Secure@Pass123', 'TK-4F2A1') -> is_authorized: True | Hidden: ('weak', 'TK-4F2A1') -> is_authorized: False

ROUND 3 — CODE RESCUE FINALE (STREAMLINED ARCHITECTURE)

Format: 1 Integrated Enterprise System | **Target:** Exactly 10 Bugs Total | **Difficulty:** Liberal / Comprehensive Architecture | **Score:** 5 Marks | **Duration:** 25 Minutes | **Code Length:** 38 Lines

Flagship Challenge: Smart Restaurant Billing & Order Dispatch System

The system powers an omnichannel restaurant ordering and dispatch engine. It validates line items against a menu catalog, calculates loyalty tier discounts and promotional coupons, computes state GST and distance-based delivery logistics fees, and orchestrates the certified customer invoice. The starter code has been **streamlined to only 38 lines of code** while containing **exactly 10 logical and runtime defects**.

Architecture: 2 Streamlined Pipeline Functions: calculate_discounted_bill → generate_order_invoice

Verification Baseline: Order: 2 Burgers (■150 ea) + 1 Pizza (■250), Tier: 'GOLD', Coupon: 'FEAST50', Distance: 5.5 km

Subtotal: ■550.0 | Discount: ■132.50 | GST (5%): ■20.88 | Delivery: ■55.00 | **Grand Total: ■493.38** (Status: CONFIRMED, Order: 'ORD-1001')

Baseline Output:

Streamlined Buggy Starter Code (Only 38 Lines | 10 Bugs Injected):

```
1 # FUNCTION 1: ITEM TOTALS & PROMO PRICING (Bugs 1 to 5)
2 def calculate_discounted_bill(order_items, menu_catalog, tier, coupon):
3     subtotal = 0.0
4     errors = []
5     for item in order_items:
6         name, qty = item.get("name"), item.get("qty", 0)
7         # BUG 1: unhandled KeyError on unknown menu item (missing 'in' check)
8         price = menu_catalog[name]
9         if qty <= 0:
10             errors.append(f"Invalid qty for {name}")
11             continue
12         # BUG 2: adds price + qty instead of multiplying
13         subtotal += price + qty
14
15     # BUG 3: strict lowercase comparison fails on uppercase 'GOLD'
16     # BUG 4: silver rate typo 0.50 (50%) instead of 0.05
17     rate = 0.20 if tier == "platinum" else (0.15 if tier == "gold" else (0.50 if tier.lower() == "silver" else 0.0))
18     discount = subtotal * rate
19     if coupon == "FEAST50" and subtotal >= 300:
20         discount += 50.0
21     discount = min(discount, subtotal * 0.50)
22
23     # BUG 5: adds discount to subtotal instead of subtracting
24     discounted_amount = subtotal + discount
25     return {"subtotal": round(subtotal, 2), "discount": round(discount, 2),
26           "discounted_amount": round(discounted_amount, 2), "errors": errors}
27
28 # FUNCTION 2: INVOICE LOGISTICS & GRAND TOTAL (Bugs 6 to 10)
29 def generate_order_invoice(order_id, order_items, menu_catalog, tier, coupon, distance_km):
30     bill = calculate_discounted_bill(order_items, menu_catalog, tier, coupon)
31     net = bill["discounted_amount"]
32     if bill["subtotal"] == 0:
33         return {"order_id": order_id, "status": "REJECTED"}
34
35     # BUG 6: integer division (5 // 100) results in 0.0 GST
36     gst = round(net * (5 // 100), 2)
37     # BUG 7: overage ignores base slab of Rs 30 for first 3 km
38     delivery = 0.0 if distance_km <= 0 else (30.0 if distance_km <= 3.0 else distance_km * 10.0)
39
40     # BUG 8: subtracts delivery fee from total instead of adding
41     grand_total = net + gst - delivery
42     # BUG 9: inverted status: sets REJECTED when grand_total > 0
43     status = "REJECTED" if grand_total > 0 else "CONFIRMED"
44
45     # BUG 10: missing 'order_id' key in invoice payload
46     return {
47         "subtotal": bill["subtotal"],
48         "discount": bill["discount"],
49         "gst": gst,
50         "delivery_fee": round(delivery, 2),
51         "grand_total": round(grand_total, 2),
52         "status": status
53     }
```

Identified Errors & Breakdown (10 Bugs Total):

- **Bug 1 (Line 8):** Direct dictionary lookup `menu_catalog[name]` raises fatal `KeyError` on unrecognized items.
- **Bug 2 (Line 13):** Computes line total as addition `price + qty` instead of multiplication `price * qty`.
- **Bug 3 (Line 17):** Tier equality compares raw lowercase `tier == 'gold'`, failing on standard uppercase tokens `'GOLD'`.
- **Bug 4 (Line 17):** Silver membership rate contains a decimal typo `0.50 (50%)` instead of `0.05 (5%)`.
- **Bug 5 (Line 23):** Computes `discounted_amount = subtotal + discount`, surcharging customers instead of deducting.
- **Bug 6 (Line 34):** GST evaluated using integer floor division `(5 // 100)`, evaluating to 0, resulting in zero tax.
- **Bug 7 (Line 36):** Overages compute `distance_km * 10.0`, omitting the required base tier of **30** for the initial 3 km.
- **Bug 8 (Line 39):** Subtracts logistics charges - delivery instead of adding logistics to payable invoice.
- **Bug 9 (Line 41):** Confirmation flag is inverted: flags orders as `'REJECTED'` when payable amount is positive.
- **Bug 10 (Lines 44-51):** Output invoice dictionary fails to return the required identification field `'order_id'`.

Complete Verified Solution — Streamlined Round 3 Engine:

```

1 def calculate_discounted_bill(order_items, menu_catalog, tier, coupon):
2     subtotal = 0.0
3     errors = []
4     for item in order_items:
5         name, qty = item.get("name"), item.get("qty", 0)
6         if name not in menu_catalog:
7             errors.append(f"Item not found: {name}")
8             continue
9         if qty <= 0:
10            errors.append(f"Invalid qty for {name}")
11            continue
12            subtotal += menu_catalog[name] * qty
13
14            t = tier.upper() if tier else "BRONZE"
15            rate = 0.20 if t == "PLATINUM" else (0.15 if t == "GOLD" else (0.05 if t == "SILVER" else 0.0))
16            discount = subtotal * rate
17            if coupon == "FEAST50" and subtotal >= 300:
18                discount += 50.0
19            discount = min(discount, subtotal * 0.50)
20            discounted_amount = subtotal - discount
21            return {"subtotal": round(subtotal, 2), "discount": round(discount, 2),
22                    "discounted_amount": round(discounted_amount, 2), "errors": errors}
23
24 def generate_order_invoice(order_id, order_items, menu_catalog, tier, coupon, distance_km):
25     bill = calculate_discounted_bill(order_items, menu_catalog, tier, coupon)
26     if bill["subtotal"] == 0:
27         return {"order_id": order_id, "status": "REJECTED"}
28
29     net = bill["discounted_amount"]
30     gst = round(net * 0.05, 2)
31     delivery = 0.0 if distance_km <= 0 else (30.0 if distance_km <= 3.0 else 30.0 + (distance_km - 3.0) * 10.0)
32
33     grand_total = net + gst + delivery
34     status = "CONFIRMED" if grand_total > 0 else "REJECTED"
35
36     return {
37         "order_id": order_id,
38         "subtotal": bill["subtotal"],
39         "discount": bill["discount"],
40         "gst": gst,
41         "delivery_fee": round(delivery, 2),
42         "grand_total": round(grand_total, 2),
43         "status": status
44     }

```

Test Baseline: Order: 2x Burger (@150) + 1x Pizza (@250) = 550.0 | GOLD (15% = 82.5) + FEAST50 = 132.50 | Net = 417.50 | GST = 20.88 | Delivery (5.5km) = 30 + 2.5*10 = 55.0 | Grand Total: **493.38**, Status: 'CONFIRMED', Order: 'ORD-1001'